

在網頁應用程式中實作密碼金鑰和表單自動填入功能

來源：<https://developers.google.com/codelabs/passkey-form-autofill?hl=zh-tw>

步驟數：7

瞭解如何在網頁應用程式中實作密碼金鑰和表單自動填入功能，讓登入過程更簡單安全。

目錄

1. 1. 事前準備
2. 2. 做好準備
3. 3. 新增建立密碼金鑰的功能
4. 4. 建構用於註冊及管理密碼金鑰憑證的 UI
5. 5. 新增透過密碼金鑰驗證的功能
6. 6. 將密碼金鑰新增至瀏覽器自動填入功能
7. 7. 恭喜！

1. 事前準備

網站使用[密碼金鑰](#)取代密碼，可讓使用者帳戶更安全、簡單且容易使用。使用者可以透過密碼金鑰，使用裝置的螢幕鎖定功能 (例如指紋、臉孔或裝置 PIN 碼) 登入網站或應用程式。如要透過密碼金鑰登入，使用者必須建立密碼金鑰並連結至使用者帳戶，再將其公開金鑰儲存在伺服器上。

在本程式碼研究室中，您會將以表單為基礎的使用者名稱和密碼登入方式，改為支援密碼金鑰，並加入下列功能：

- 使用者登入後，系統會顯示按鈕，讓使用者建立密碼金鑰。
- 顯示已註冊密碼金鑰清單的 UI。
- 現有的登入表單，可讓使用者透過表單自動填入功能，使用已註冊的密碼金鑰登入。

注意：本程式碼研究室不會教您如何建構 FIDO 伺服器，而是使用名為 [SimpleWebAuthn](#) 的程式庫。使用第三方程式庫時，請確認程式庫提供所需功能。如要進一步瞭解其他 FIDO 伺服器選項，請參閱「[FIDO 認證](#)」。如要進一步瞭解開放原始碼程式庫，請參閱「[WebAuthn and Passkey Awesome](#)」。

必要條件

- 對 JavaScript 有基本瞭解
- [Web Authentication API \(WebAuthn\)](#) 的基本概念

課程內容

- 如何建立密碼金鑰
 - 如何使用密碼金鑰驗證使用者。
 - 如何讓表單建議使用密碼金鑰做為登入選項。
-

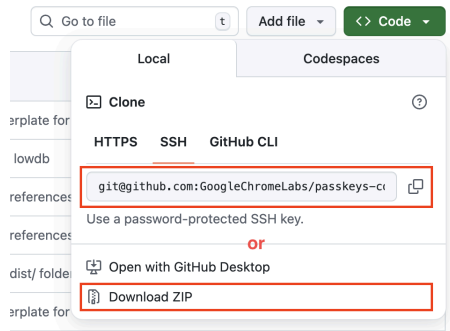
步驟 2 · 約 2 分鐘

2. 做好準備

在本程式碼研究室中，您將從 GitHub 複製未完成的試用版應用程式，然後完成密碼金鑰支援的實作。

複製專案

1. 在 GitHub 上開啟[專案](#)。
2. [複製](#)或下載專案。



執行專案

1. 開啟終端機並 `cd start` 變更目錄。
2. 執行 `npm install` 安裝專案依附元件。
3. 使用 `npm run build && IS_LOCAL=1 npm run start` 建構並執行專案。
4. 在瀏覽器中開啟 <http://localhost:8080/>。

檢查網站的初始狀態

1. 在網站上輸入隨機使用者名稱，然後按一下「下一步」。
2. 輸入隨機密碼，然後按一下「登入」。系統會忽略密碼，但您仍會通過驗證並前往首頁。
3. 如要變更顯示名稱，請變更。在初始狀態下，您只能執行這項操作。
4. 按一下「登出」。

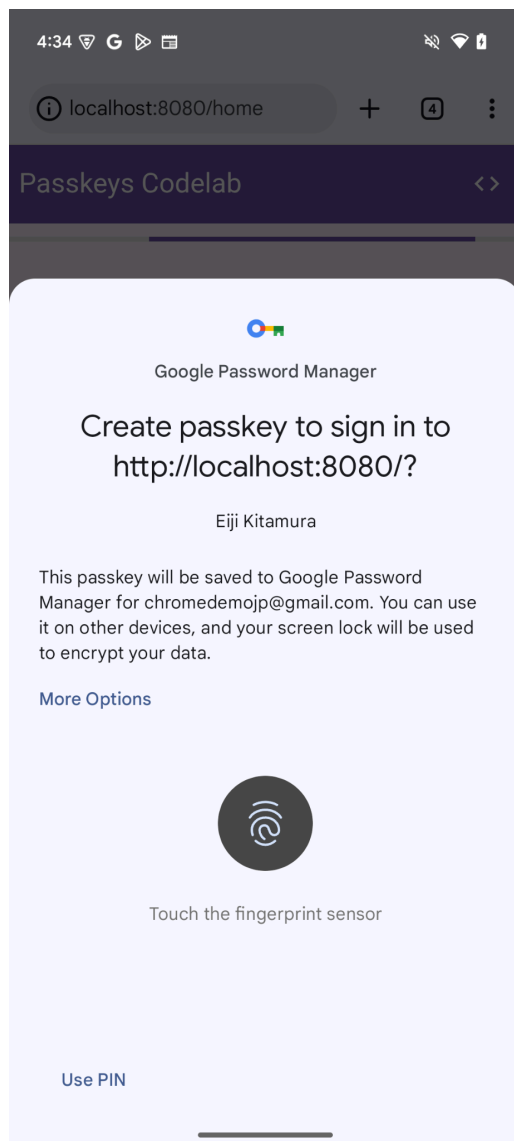
在此狀態下，使用者每次登入時都必須輸入密碼。您會在這個表單中新增密碼金鑰支援功能，讓使用者透過裝置的螢幕鎖定功能登入。

如要進一步瞭解密碼金鑰的運作方式，請參閱[這篇文章](#)。

步驟 3 · 約 10 分鐘

3. 新增建立密碼金鑰的功能

如要讓使用者透過密碼金鑰驗證身分，您必須允許他們建立及註冊密碼金鑰，並將公開金鑰儲存在伺服器上。



您希望允許使用者在以密碼登入後建立密碼金鑰，並新增 UI，讓使用者在 `/home` 頁面上建立密碼金鑰及查看所有已註冊的密碼金鑰清單。在下一節中，您將建立用於建立及註冊密碼金鑰的函式。

建立 `registerCredential()` 函式

1. 在您選擇的程式碼編輯器中，開啟 `start` 目錄。
2. 前往 `public/client.js` 檔案，然後捲動至結尾。
3. 在相關註解後方新增下列 `registerCredential()` 函式：

`public/client.js`

```
// TODO: Add an ability to create a passkey: Create the registerCredential() function.
export async function registerCredential() {

  // TODO: Add an ability to create a passkey: Obtain the challenge and other options from
  the server endpoint.

  // TODO: Add an ability to create a passkey: Create a credential.

  // TODO: Add an ability to create a passkey: Register the credential to the server
  endpoint.

};
```

這項函式會在伺服器上建立及註冊密碼金鑰。

從伺服器端點取得驗證問題和其他選項

在建立密碼金鑰前，你必須要求從伺服器傳遞 [WebAuthn](#) 形式的參數，包括驗證問題。WebAuthn 是一種瀏覽器 API，可讓使用者建立密碼金鑰，並透過密碼金鑰驗證使用者。幸好在本程式碼研究室中，您已有會傳回這類參數的伺服器端點。

- 如要從伺服器端點取得驗證碼和其他選項，請在相關註解後方，將下列程式碼新增至 `registerCredential()` 函式主體：

public/client.js

```
// TODO: Add an ability to create a passkey: Obtain the challenge and other options from the
server endpoint.
const _options = await _fetch('/auth/registerRequest');
```

注意：本程式碼研究室中的 `_fetch()` 函式已預先定義為 `POST` 方法，且 `Content-Type` HTTP 要求標頭已設為 `application/json` 值。並剖析及傳回伺服器傳回的 JSON。

以下程式碼片段包含伺服器提供的範例選項：

```

{
  challenge: *****,
  rp: {
    id: "example.com",
  },
  user: {
    id: *****,
    name: "john78",
    displayName: "John",
  },
  pubKeyCredParams: [{
    alg: -7, type: "public-key"
  }, {
    alg: -257, type: "public-key"
  }],
  excludeCredentials: [{
    id: *****,
    type: 'public-key',
    transports: ['internal', 'hybrid'],
  }],
  authenticatorSelection: {
    authenticatorAttachment: "platform",
    requireResidentKey: true,
  }
}

```

伺服器與用戶端之間的通訊協定不屬於 [WebAuthn 規格](#)。不過，本程式碼研究室的伺服器會盡可能傳回與傳遞至 WebAuthn `navigator.credentials.create()` API 的 [PublicKeyCredentialCreationOptions](#) 字典類似的 JSON。

下表僅列出部分內容，但包含 `PublicKeyCredentialCreationOptions` 字典中的重要參數：

參數	說明
<code>challenge</code>	這個註冊程序的 <code>ArrayBuffer</code> 物件中，伺服器產生的驗證問題。這是必填欄位，但在註冊期間不會用到 (除非要進行驗證)。驗證是進階主題，本程式碼研究室不會介紹。
<code>user.id</code>	使用者的專屬 ID。這個值必須是 <code>ArrayBuffer</code> 物件，且不得包含個人身分資訊，例如電子郵件地址或使用者名稱。你也可以使用系統為各個帳戶產生的隨機 16 位元組值。
<code>user.name</code>	這個欄位應包含使用者能夠認得的帳戶專屬 ID，例如電子郵件地址或使用者名稱。這會顯示在帳戶選取器中 (如要指定使用者名稱，請使用密碼驗證中的值)。
<code>user.displayName</code>	此為選填欄位，用於指定容易辨識的帳戶名稱。這個名稱可以重複，也可以是使用者選擇的名稱。如果網站沒有適合的值可在此處加入，請傳遞空字串。視瀏覽器而定，這項資訊可能會顯示在帳戶選取器中。
<code>rp.id</code>	信賴憑證者 (RP) ID 是網域。網站可指定其網域或 可註冊的後置字串 。舉例來說，如果 RP 的來源是 <code>https://login.example.com:1337</code> ，RP ID 可以是 <code>login.example.com</code> 或 <code>example.com</code> 。如果 RP ID

	指定為 <code>example.com</code> ，使用者可以在 <code>login.example.com</code> 或 <code>example.com</code> 的任何其他子網域上進行驗證。
<code>pubKeyCredParams</code>	這個欄位會指定 RP 支援的公開金鑰演算法。建議您將這項設定設為 <code>[{alg: -7, type: "public-key"}, {alg: -257, type: "public-key"}]</code> 。這會指定支援使用 P-256 和 RSA PKCS#1 的 ECDSA，支援這些項目可確保完整涵蓋範圍。
<code>excludeCredentials</code>	提供已註冊憑證 ID 清單，避免重複註冊同一部裝置。如果提供 <code>transports</code> 成員，該成員應包含註冊各個憑證時呼叫 <code>getTransports()</code> 函式的結果。如要瞭解詳情，請參閱說明文件，瞭解如何 避免在已有密碼金鑰時建立新密碼金鑰 。
<code>authenticatorSelection.authenticatorAttachment</code>	設為 <code>"platform"</code> 值。這表示您想將驗證器嵌入平台裝置，因此系統不會提示使用者插入 USB 安全金鑰等。
<code>authenticatorSelection.requireResidentKey</code>	設為布林 <code>true</code> 值。可探索的憑證 (常駐金鑰) 無須伺服器提供憑證 ID 即可使用，因此與自動填入功能相容。詳情請參閱 可探索的憑證深入探討 。
<code>authenticatorSelection.userVerification</code>	設為 <code>"preferred"</code> 值或省略，因為這是預設值。這表示使用裝置螢幕鎖定的使用者驗證是 <code>"required"</code> 、 <code>"preferred"</code> 或 <code>"discouraged"</code> 。如果設為 <code>"preferred"</code> 值，系統會在裝置支援時要求使用者驗證身分。詳情請參閱 使用者驗證深入探索 。

注意：如果裝置不支援生物辨識感應器 (例如 iMac)，將 `authenticatorSelection.userVerification` 參數設為 `"preferred"` 值，系統就不會要求使用者驗證身分，並立即傳回含有 [UV \(使用者驗證\)](#) 旗標和 `false` 值的憑證。這是本程式碼研究室的預期行為，因為伺服器不需要 UV 旗標，如 [libs/auth.mjs](#) 檔案所示。

如要一律要求使用者驗證，請將 `authenticatorSelection.userVerification` 屬性設為 `"required"` 值，並將 [libs/auth.mjs](#) 檔案中的行替換為設為 `true` 值的 `requireUserVerification` 屬性。

注意：部分瀏覽器不需要 `authenticatorSelection` 字典中的任何參數即可建立密碼金鑰，但其他瀏覽器可能需要。建議您明確指定這些參數。

建立憑證

- 在 `registerCredential()` 函式主體中相關註解後方，將以 Base64URL 編碼的部分參數轉換回二進位，具體來說是 `user.id` 和 `challenge` 字串，以及 `excludeCredentials` 陣列中包含的 `id` 字串例項。您可以使用 `PublicKeyCredential.parseCreationOptionsFromJSON()` 函式執行這項操作：

[public/client.js](#)

```
// TODO: Add an ability to create a passkey: Create a credential.

// Deserialize and decode the `PublicKeyCredential.parseCreationOptionsFromJSON()`.
const options = PublicKeyCredential.parseCreationOptionsFromJSON(_options);
```

2. 在下一行中，將 `authenticatorSelection.authenticatorAttachment` 設為 `"platform"`，並將 `authenticatorSelection.requireResidentKey` 設為 `true`。這樣一來，使用者只能透過具備[可探索憑證功能](#)的平台驗證器 (裝置本身) 進行驗證。

注意：Android 版 Google 密碼管理工具可以建立可供偵測和不可供偵測的憑證，但只會同步處理可供偵測的憑證。Apple 平台上的 iCloud 鑰匙圈只會建立可供偵測的憑證，且一律會同步處理。

public/client.js

```
// Use platform authenticator and discoverable credential.
options.authenticatorSelection = {
  authenticatorAttachment: 'platform',
  requireResidentKey: true
}
```

3. 在下一行呼叫 `navigator.credentials.create()` 方法，建立憑證。

public/client.js

```
// Invoke the WebAuthn create() method.
const cred = await navigator.credentials.create({
  publicKey: options,
});
```

透過這項呼叫，瀏覽器會嘗試[使用裝置的螢幕鎖定功能驗證使用者身分](#)。

注意：就算使用者透過生物特徵辨識感應器建立密碼金鑰，伺服器也絕對無法存取生物特徵辨識資料。裝置只會確認已儲存的生物特徵辨識資訊是否與使用者的本機資訊一致，藉此建立密碼金鑰或使用密碼金鑰登入。生物特徵辨識資訊只會儲存在裝置上，絕不會外洩。

向伺服器端點註冊憑證

使用者驗證身分後，系統會建立並儲存密碼金鑰。網站會收到包含公開金鑰的憑證物件，你可以將該金鑰傳送至伺服器來註冊密碼金鑰。

以下程式碼片段包含憑證物件範例：

```

{
  "id": *****,
  "rawId": *****,
  "type": "public-key",
  "response": {
    "clientDataJSON": *****,
    "attestationObject": *****,
    "transports": ["internal", "hybrid"]
  },
  "authenticatorAttachment": "platform"
}

```

下表僅列出部分內容，但包含 `PublicKeyCredential` 物件中的重要參數：

參數	說明
<code>id</code>	所建立密碼金鑰的 Base64URL 編碼 ID。這個 ID 可在驗證時，協助瀏覽器判斷裝置上是否有相符的密碼金鑰。這個值必須儲存在後端資料庫中。
<code>rawId</code>	憑證 ID 的 <code>ArrayBuffer</code> 物件版本。
<code>response.clientDataJSON</code>	<code>ArrayBuffer</code> 物件編碼的用戶端資料。
<code>response.attestationObject</code>	<code>ArrayBuffer</code> 編碼的認證物件，當中包含重要資訊，例如 RP ID、旗標和公開金鑰。
<code>response.transports</code>	裝置支援的傳輸方式清單： <code>"internal"</code> 表示裝置支援密碼金鑰。 <code>"hybrid"</code> 表示裝置也支援在 其他裝置上進行驗證 。
<code>authenticatorAttachment</code>	如果是在支援密碼金鑰的裝置上建立這項憑證，則傳回 <code>"platform"</code> 。

注意：憑證的 `response` 成員 (`AuthenticatorAttestationResponse` 物件) 會提供 `getTransports()`、`getPublicKey()`、`getAuthenticatorData()` 和 `getPublicKeyAlgorithm()` 函式，用於擷取憑證元素。

如要將憑證物件傳送至伺服器，請按照下列步驟操作：

1. 將憑證的二進位參數編碼為 Base64URL，以便以字串形式傳送至伺服器。您可以使用 `.toJSON()` 執行這項操作：

public/client.js

```
// TODO: Add an ability to create a passkey: Register the credential to the server endpoint.  
  
// Encode and serialize the `PublicKeyCredential`.  
const credential = JSON.stringify(cred);
```

2. 在下一行中，將物件傳送至伺服器：

[public/client.js](#)

```
return await _fetch('/auth/registerResponse', credential);
```

執行程式時，伺服器會傳回 `HTTP code 200`，表示憑證已註冊。

現在您已擁有完整的 `registerCredential()` 函式！

查看本節的解決方案程式碼

[public/client.js](#)

```
// TODO: Add an ability to create a passkey: Create the registerCredential() function.

export async function registerCredential() {

  // TODO: Add an ability to create a passkey: Obtain the challenge and other options from
  the server endpoint.

  const _options = await _fetch('/auth/registerRequest');

  // TODO: Add an ability to create a passkey: Create a credential.

  // Deserialize and decode the `PublicKeyCredential.parseCreationOptionsFromJSON()`.
  const options = PublicKeyCredential.parseCreationOptionsFromJSON(_options);

  // Use platform authenticator and discoverable credential.
  options.authenticatorSelection = {
    authenticatorAttachment: 'platform',
    requireResidentKey: true
  }

  // Invoke the WebAuthn create() method.
  const cred = await navigator.credentials.create({
    publicKey: options,
  });

  // TODO: Add an ability to create a passkey: Register the credential to the server
  endpoint.

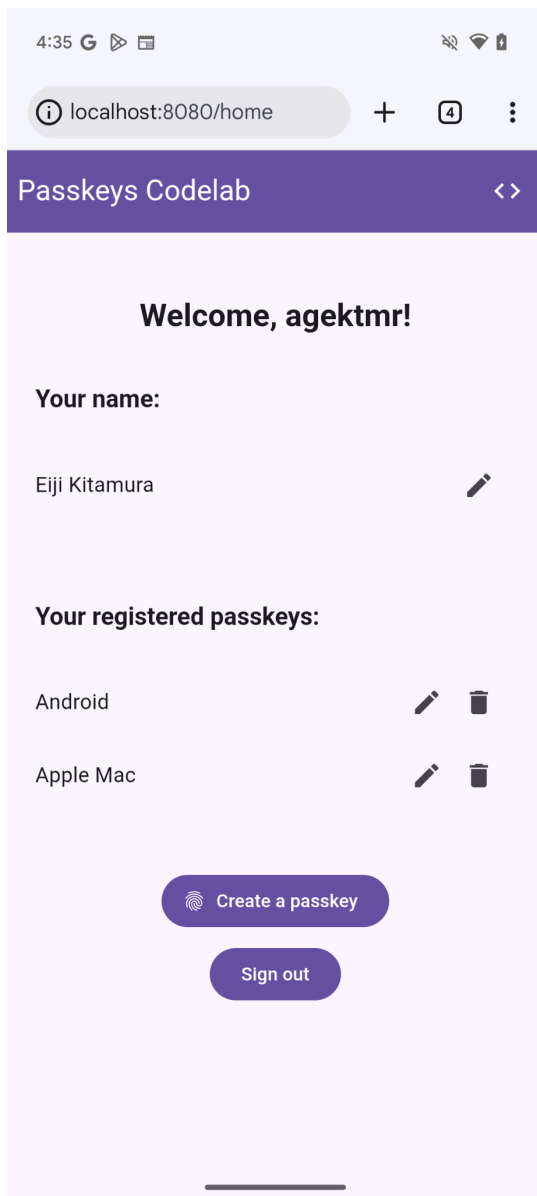
  // Encode and serialize the `PublicKeyCredential`.
  const credential = JSON.stringify(cred);

  return await _fetch('/auth/registerResponse', credential);
};
```

步驟 4 · 約 8 分鐘

4. 建構用於註冊及管理密碼金鑰憑證的 UI

現在 `registerCredential()` 函式已可使用，您需要一個按鈕來叫用該函式。此外，您還需要顯示已註冊的密碼金鑰清單。



新增預留位置 HTML

1. 在編輯器中，前往 `views/home.html` 檔案。
2. 在相關註解後方，新增 UI 預留位置，顯示註冊密碼金鑰的按鈕和密碼金鑰清單：

[views/home.html](#)

```

<!-- TODO: Add an ability to create a passkey: Add placeholder HTML. -->
<section>
  <h3>Your registered passkeys:</h3>
  <div id="list"></div>
</section>
<p id="message" class="instructions"></p>
<mdui-button id="create-passkey" class="hidden" icon="fingerprint" type="button">Create a
passkey</mdui-button>

```

`div#list` 元素是清單的預留位置。

檢查是否支援密碼金鑰

如要只向裝置支援密碼金鑰的使用者顯示建立密碼金鑰的選項，請先檢查 WebAuthn 是否可用。如果是，您必須移除 `hidden` 類別，才能顯示「建立密碼金鑰」按鈕。

如要檢查環境是否支援密碼金鑰，請按照下列步驟操作：

1. 在 `views/home.html` 檔案結尾處的相關註解後方，編寫條件，在 `window.PublicKeyCredential`、`PublicKeyCredential.isUserVerifyingPlatformAuthenticatorAvailable` 和 `PublicKeyCredential.isConditionalMediationAvailable` 為 `true` 時執行。

views/home.html

```

// TODO: Add an ability to create a passkey: Check for passkey support.
const createPasskey = $('#create-passkey');
// Feature detections
if (window.PublicKeyCredential &&
    PublicKeyCredential.isUserVerifyingPlatformAuthenticatorAvailable &&
    PublicKeyCredential.isConditionalMediationAvailable) {

```

2. 在條件式的主體中，檢查裝置是否可以建立密碼金鑰，然後檢查是否可以在表單自動填入功能中建議密碼金鑰。

views/home.html

```

try {
  const capabilities = await PublicKeyCredential.getClientCapabilities();
  // Is conditional UI available in this browser?
  if (capabilities.conditionalGet === true &&
      capabilities.passkeyPlatformAuthenticator === true) {

```

3. 如果符合所有條件，系統就會顯示建立密碼金鑰的按鈕。否則，請顯示警告訊息。

views/home.html

```

    createPasskey.classList.remove('hidden');
  } else {

    // If conditional UI isn't available, show a message.
    $('#message').innerText = 'This device does not support passkeys.';
  }
} catch (e) {
  console.error(e);
}
} else {

  // If WebAuthn isn't available, show a message.
  $('#message').innerText = 'This device does not support passkeys.';
}

```

在清單中顯示已註冊的密碼金鑰

1. 定義 `renderCredentials()` 函式，從伺服器擷取已註冊的密碼金鑰，並以清單形式呈現。幸好您已有 `/auth/getKeys` 伺服器端點，可擷取已登入使用者的已註冊密碼金鑰。

views/home.html

```

// TODO: Add an ability to create a passkey: Render registered passkeys in a list.
async function renderCredentials() {
  const res = await _fetch('/auth/getKeys');
  const list = $('#list');
  const creds = res.length > 0 ? html`
    <mdui-list>
      ${res.map(cred => html`
        <mdui-list-item>
          ${cred.name || 'Unnamed'}
          <mdui-button-icon data-cred-id="${cred.id}" data-name="${cred.name || 'Unnamed'}"
@click="${rename}" icon="edit" slot="end-icon"></mdui-button-icon>
          <mdui-button-icon data-cred-id="${cred.id}" @click="${remove}" icon="delete"
slot="end-icon"></mdui-button-icon>
        </mdui-list-item>`)}`
    : html`
    <mdui-list>
      <mdui-list-item>No credentials found.</mdui-list-item>
    </mdui-list>`;
  render(creds, list);
};

```

2. 在下一行中，叫用 `renderCredentials()` 函式，在使用者抵達 `/home` 頁面時立即顯示已註冊的密碼金鑰，做為初始化程序。

views/home.html

```
renderCredentials();
```

建立及註冊密碼金鑰

如要建立及註冊密碼金鑰，請呼叫您先前實作的 `registerCredential()` 函式。

如要在點選「建立密碼金鑰」按鈕時觸發 `registerCredential()` 函式，請按照下列步驟操作：

1. 在預留位置 HTML 後方的 檔案中，找出下列 `import` 陳述式：

views/home.html

```
import {
  $,
  _fetch,
  loading,
  updateCredential,
  unregisterCredential,
} from '/client.js';
```

2. 在 `import` 陳述式主體的結尾，新增 `registerCredential()` 函式。

views/home.html

```
// TODO: Add an ability to create a passkey: Create and register a passkey.
import {
  $,
  _fetch,
  loading,
  updateCredential,
  unregisterCredential,
  registerCredential
} from '/client.js';
```

3. 在檔案結尾的相關註解後方，定義 `register()` 函式，該函式會叫用 `registerCredential()` 函式和載入 UI，並在註冊後呼叫 `renderCredentials()`。這項說明指出，瀏覽器會建立密碼金鑰，並在發生錯誤時顯示錯誤訊息。

views/home.html

```
// TODO: Add an ability to create a passkey: Create and register a passkey.
async function register() {
  try {

    // Start the loading UI.
    loading.start();

    // Start creating a passkey.
    await registerCredential();

    // Stop the loading UI.
    loading.stop();

    // Render the updated passkey list.
    renderCredentials();
  }
}
```

4. 在 `register()` 函式主體中，擷取例外狀況。如果裝置上已有密碼金鑰，`navigator.credentials.create()` 方法會擲回 `InvalidStateError` 錯誤。這會透過 `excludeCredentials` 陣列檢查。在本例中，您會向使用者顯示相關訊息。使用者取消驗證對話方塊時，也會擲回 `NotAllowedError` 錯誤。在這種情況下，您會以無聲方式忽略該要求。

views/home.html

```
} catch (e) {

  // Stop the loading UI.
  loading.stop();

  // An InvalidStateError indicates that a passkey already exists on the device.
  if (e.name === 'InvalidStateError') {
    alert('A passkey already exists for this device.');
```

```
    // A NotAllowedError indicates that the user canceled the operation.
  } else if (e.name === 'NotAllowedError') {
    Return;

    // Show other errors in an alert.
  } else {
    alert(e.message);
    console.error(e);
  }
}
};
```

5. 在 `register()` 函式後方的行中，將 `register()` 函式附加至「建立密碼金鑰」按鈕的 `click` 事件。

views/home.html

```
createPasskey.addEventListener('click', register);
```

查看本節的解決方案程式碼

views/home.html

```
<!-- TODO: Add an ability to create a passkey: Add placeholder HTML. -->
<section>
  <h3>Your registered passkeys:</h3>
  <div id="list"></div>
</section>
<p id="message" class="instructions"></p>
<mdui-button id="create-passkey" icon="fingerprint" type="button">Create a passkey</mdui-button>
```

views/home.html

```
// TODO: Add an ability to create a passkey: Create and register a passkey.
import {
  $,
  _fetch,
  loading,
  updateCredential,
  unregisterCredential,
  registerCredential
} from '/client.js';
```

views/home.html

```

// TODO: Add an ability to create a passkey: Check for passkey support.
const createPasskey = $('#create-passkey');

// Is WebAuthn available in this browser?
if (window.PublicKeyCredential &&
    PublicKeyCredential.isUserVerifyingPlatformAuthenticatorAvailable &&
    PublicKeyCredential.isConditionalMediationAvailable) {
  try {
    const capabilities = await PublicKeyCredential.getClientCapabilities();
    // Is conditional UI available in this browser?
    if (capabilities.conditionalGet === true &&
        capabilities.passkeyPlatformAuthenticator === true) {
      // If conditional UI is available, reveal the Create a passkey button.
      createPasskey.classList.remove('hidden');
    } else {
      // If conditional UI isn't available, show a message.
      $('#message').innerText = 'This device does not support passkeys.';
    }
  } catch (e) {
    console.error(e);
  }
} else {
  // If WebAuthn isn't available, show a message.
  $('#message').innerText = 'This device does not support passkeys.';
}

// TODO: Add an ability to create a passkey: Render registered passkeys in a list.

async function renderCredentials() {
  const res = await _fetch('/auth/getKeys');
  const list = $('#list');
  const creds = html`${res.length > 0 ? html`
    <mdui-list>
      ${res.map(cred => html`
        <mdui-list-item>
          ${cred.name || 'Unnamed'}
          <mdui-button-icon data-cred-id="${cred.id}" data-name="${cred.name || 'Unnamed'}"
@click="${cred.rename}" icon="edit" slot="end-icon"></mdui-button-icon>
          <mdui-button-icon data-cred-id="${cred.id}" @click="${cred.remove}" icon="delete"
slot="end-icon"></mdui-button-icon>
        </mdui-list-item>`)}
    </mdui-list>` : html`
    <mdui-list>
      <mdui-list-item>No credentials found.</mdui-list-item>
    </mdui-list>`}`;
  render(creds, list);
};

renderCredentials();

// TODO: Add an ability to create a passkey: Create and register a passkey.

```

```
async function register() {
  try {
    // Start the loading UI.
    loading.start();
    // Start creating a passkey.
    await registerCredential();
    // Stop the loading UI.
    loading.stop();
    // Render the updated passkey list.
    renderCredentials();
  } catch (e) {
    // Stop the loading UI.
    loading.stop();
    // An InvalidStateError indicates that a passkey already exists on the device.
    if (e.name === 'InvalidStateError') {
      alert('A passkey already exists for this device.');
```

// A NotAllowedError indicates the user canceled the operation.

```
    } else if (e.name === 'NotAllowedError') {
      return;
      // Show other errors in an alert.
    } else {
      alert(e.message);
      console.error(e);
    }
  }
};

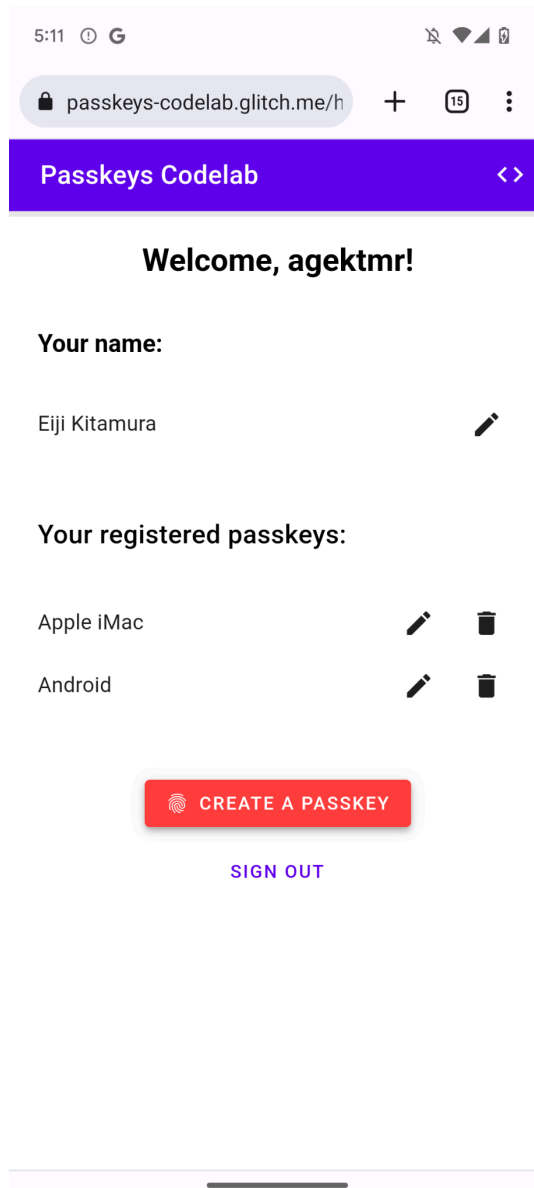
createPasskey.addEventListener('click', register);
```

立即體驗

如果您已完成所有步驟，就代表您已在網站上導入建立、註冊及顯示密碼金鑰的功能！

如要試用這項功能，請按照以下步驟操作：

1. 在該網站上，使用隨機使用者名稱和密碼登入。
2. 按一下「建立密碼金鑰」。
3. 使用裝置的螢幕鎖定功能驗證身分。
4. 確認網頁的「已註冊的密碼金鑰」部分下方顯示已註冊的密碼金鑰。



注意：密碼金鑰名稱是從使用者代理程式字串的 `platform` 衍生而來。

重新命名及移除已註冊的密碼金鑰

您應該可以重新命名或刪除清單中的已註冊密碼金鑰。您可以查看程式碼研究室隨附的程式碼，瞭解其運作方式。

在 Chrome 中，您可以透過桌機的 `chrome://settings/passkeys` 或 Android 裝置設定中的密碼管理工具，移除已註冊的密碼金鑰。

如要瞭解如何在其他平台上重新命名及移除已註冊的密碼金鑰，請參閱這些平台的支援頁面。

步驟 5 · 約 8 分鐘

5. 新增透過密碼金鑰驗證的功能

使用者現在可以建立及註冊密碼金鑰，並使用密碼金鑰或密碼，安全地進行網站驗證。現在您需要在網站中新增密碼金鑰驗證功能。

建立 `authenticate()` 函式

- 在 `public/client.js` 檔案中相關註解之後，建立名為 `authenticate()` 的函式，在本地驗證使用者，然後對伺服器進行驗證：

`public/client.js`

```
// TODO: Add an ability to authenticate with a passkey: Create the authenticate() function.
export async function authenticate() {

  // TODO: Add an ability to authenticate with a passkey: Obtain the challenge and other
  options from the server endpoint.

  // TODO: Add an ability to authenticate with a passkey: Locally verify the user and get a
  credential.

  // TODO: Add an ability to authenticate with a passkey: Verify the credential.

};
```

從伺服器端點取得驗證問題和其他選項

在要求使用者進行驗證前，你必須要求從伺服器傳遞 WebAuthn 參數，包括驗證問題。

- 在相關註解後方的 `authenticate()` 函式主體中，呼叫 `_fetch()` 函式，將 `POST` 要求傳送至伺服器：

`public/client.js`

```
// TODO: Add an ability to authenticate with a passkey: Obtain the challenge and other
options from the server endpoint.

// Base64URL decode the challenge.
const options = PublicKeyCredential.parseRequestOptionsFromJSON(_options);
```

本程式碼研究室的伺服器會盡可能傳回與傳遞至 WebAuthn `navigator.credentials.get()` API 的 `PublicKeyCredentialRequestOptions` 字典類似的 JSON。以下程式碼片段包含所提供的範例選項：

```
{
  "challenge": "*****",
  "rpId": "localhost",
  "allowCredentials": []
}
```

下表僅列出部分內容，但包含 `PublicKeyCredentialRequestOptions` 字典中的重要參數：

參數	說明
<code>challenge</code>	<code>ArrayBuffer</code> 物件中伺服器產生的驗證問題。你必須提供這個值來防範重送攻擊。切勿在單一回應中接受相同驗證問題兩次。
<code>rpId</code>	RP ID 是網域。網站可指定其網域或可註冊的後置字串。這個值必須與建立密碼金鑰時使用的 <code>rp.id</code> 參數一致。
<code>allowCredentials</code>	這項屬性用於尋找符合這項驗證資格的驗證器。傳遞空陣列或不指定，讓瀏覽器顯示帳戶選取器。進一步瞭解 <code>allowCredentials</code> 的行為。
<code>userVerification</code>	設為 "preferred" 值或省略，因為這是預設值。這表示使用裝置螢幕鎖定進行使用者驗證的狀態為 "required"、"preferred" 或 "discouraged"。如果設為 "preferred" 值，系統會在裝置支援時要求使用者驗證身分。進一步瞭解 使用者驗證行為 。

在本機驗證使用者並取得憑證

1. 在 `authenticate()` 函式主體中，於相關註解後方將 `challenge` 參數轉換回二進位：

public/client.js

```
// TODO: Add an ability to authenticate with a passkey: Locally verify the user and get a credential.
// Base64URL decode the challenge.
options.challenge = base64url.decode(options.challenge);
```

2. 將空陣列傳遞至 `allowCredentials` 參數，在使用者驗證時開啟帳戶選取器：

public/client.js

```
// An empty allowCredentials array invokes an account selector by discoverable credentials.
options.allowCredentials = [];
```

帳戶選取器會使用密碼金鑰儲存的使用者資訊。

注意：在密碼金鑰建立作業中傳遞 `requireResidentKey: true` 布林值，即可啟用可探索的憑證，並儲存使用者資訊 (例如名稱和顯示名稱)，這些資訊會在驗證後顯示在帳戶選單中。

3. 呼叫 `navigator.credentials.get()` 方法，並提供 `mediation: 'conditional'` 選項：

public/client.js

```
// Invoke the WebAuthn get() method.
const cred = await navigator.credentials.get({
  publicKey: options,

  // Request a conditional UI.
  mediation: 'conditional'
});
```

這個選項會指示瀏覽器在表單自動填入時，有條件地建議使用密碼金鑰。

驗證憑證

使用者在本地驗證身分後，您應該會收到含有簽章的憑證物件，您可以在伺服器上驗證該簽章。

以下程式碼片段包含範例 `PublicKeyCredential` 物件：

```
{
  "id": *****,
  "rawId": *****,
  "type": "public-key",
  "response": {
    "clientDataJSON": *****,
    "authenticatorData": *****,
    "signature": *****,
    "userHandle": *****
  },
  authenticatorAttachment: "platform"
}
```

下表僅列出部分內容，但包含 `PublicKeyCredential` 物件中的重要參數：

參數	說明
<code>id</code>	經驗證密碼金鑰憑證的 Base64URL 編碼 ID。
<code>rawId</code>	憑證 ID 的 <code>ArrayBuffer</code> 物件版本。
<code>response.clientDataJSON</code>	用戶端資料的 <code>ArrayBuffer</code> 物件。這個欄位包含多項資訊，例如驗證問題和 RP 伺服器必須驗證的來源。
<code>response.authenticatorData</code>	驗證器資料的 <code>ArrayBuffer</code> 物件。這個欄位包含 RP ID 等資訊。
<code>response.signature</code>	簽名的 <code>ArrayBuffer</code> 物件。這個值是驗證作業的核心元素，必須在伺服器上進行驗證。

<code>response.userHandle</code>	<code>ArrayBuffer</code> 物件，當中包含建立時設定的使用者 ID。如果伺服器必須選擇要使用的 ID 值，或是後端想避免建立憑證 ID 索引，則可使用這個值而非憑證 ID。
<code>authenticatorAttachment</code>	如果憑證來自本機裝置，則傳回 <code>"platform"</code> 字串。否則會傳回 <code>"cross-platform"</code> 字串，特別是使用者透過手機登入時。如果使用者需要透過手機登入，請提示他們在當地裝置上建立密碼金鑰。

如要將憑證物件傳送至伺服器，請按照下列步驟操作：

1. 在 `authenticate()` 函式主體中，於相關註解後方編碼憑證的二進位參數，以便將其做為字串傳送至伺服器。您可以使用 `.toJSON()` 執行這項操作：

public/client.js

```
// TODO: Add an ability to authenticate with a passkey: Verify the credential.
// Encode and serialize the `PublicKeyCredential`.
const credential = JSON.stringify(cred);
```

2. 將物件傳送至伺服器：

public/client.js

```
return await _fetch(`/auth/signinResponse`, credential);
```

執行程式時，伺服器會傳回 `HTTP code 200`，表示憑證已通過驗證。

注意：伺服器必須驗證憑證，才能驗證使用者。不過，在本程式碼研究室中，您不會學到如何在伺服器端執行這些驗證。請勿將本程式碼研究室中的程式碼複製到正式環境。如要瞭解第三方解決方案，請前往 [FIDO 聯盟官方頁面](#)；如要瞭解開放原始碼程式庫，請前往 [WebAuthn and Passkey Awesome](#)。

您現在已擁有完整的 `authentication()` 函式！

查看本節的解決方案程式碼

public/client.js

```

// TODO: Add an ability to authenticate with a passkey: Create the authenticate() function.
export async function authenticate() {

  // TODO: Add an ability to authenticate with a passkey: Obtain the
  challenge and other options from the server endpoint.
  const options = await _fetch('/auth/signinRequest');

  // TODO: Add an ability to authenticate with a passkey: Locally verify
  the user and get a credential.
  // Base64URL decode the challenge.
  options.challenge = base64url.decode(options.challenge);

  // The empty allowCredentials array invokes an account selector
  by discoverable credentials.
  options.allowCredentials = [];

  // Invoke the WebAuthn get() function.
  const cred = await navigator.credentials.get({
    publicKey: options,

    // Request a conditional UI.
    mediation: 'conditional'
  });

  // TODO: Add an ability to authenticate with a passkey: Verify the credential.
  const credential = {};
  credential.id = cred.id;
  credential.rawId = cred.id; // Pass a Base64URL encoded ID string.
  credential.type = cred.type;

  // Base64URL encode some values.
  const clientDataJSON = base64url.encode(cred.response.clientDataJSON);
  const authenticatorData =
    base64url.encode(cred.response.authenticatorData);
  const signature = base64url.encode(cred.response.signature);
  const userHandle = base64url.encode(cred.response.userHandle);

  credential.response = {
    clientDataJSON,
    authenticatorData,
    signature,
    userHandle,
  };

  return await _fetch(`/auth/signinResponse`, credential);
};

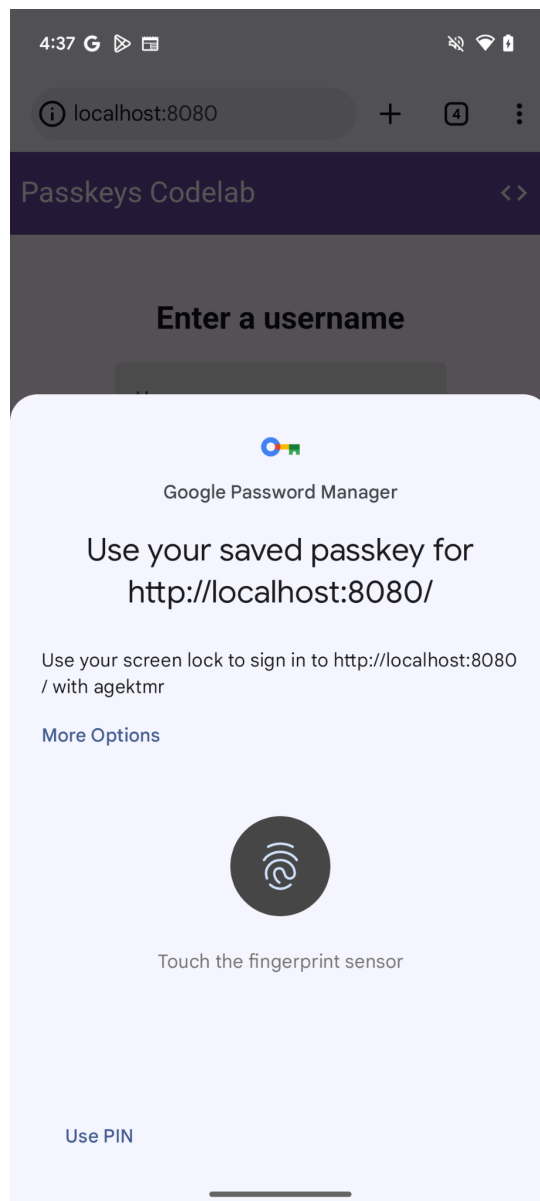
```

6. 將密碼金鑰新增至瀏覽器自動填入功能

使用者返回時，您希望他們盡可能輕鬆安全地登入。如果在登入頁面新增「使用密碼金鑰登入」按鈕，使用者就能按下該按鈕，在瀏覽器的帳戶選取器中選取密碼金鑰，並使用螢幕鎖定驗證身分。

不過，並非所有使用者都會立即從密碼改用密碼金鑰。也就是說，您必須等到所有使用者都改用密碼金鑰，才能移除密碼，因此您必須保留以密碼為基礎的登入表單。不過，如果您同時保留密碼表單和密碼金鑰按鈕，使用者就必須在兩者之間做出不必要的選擇。理想情況下，您會希望登入程序簡單明瞭。

這時就需要條件式 UI。條件式 UI 是 WebAuthn 功能，可讓您在表單輸入欄位中，除了密碼之外，也建議將密碼金鑰做為自動填入項目。如果使用者輕觸自動填入建議中的密碼金鑰，系統會要求使用者使用裝置的螢幕鎖定功能，在裝置上驗證身分。使用者動作幾乎與密碼登入相同，因此可享有流暢的使用體驗。



啟用條件式 UI

如要啟用條件式 UI，只要在輸入欄位的 `autocomplete` 屬性中新增 `webauthn` 權杖即可。設定權杖後，您可以使用 `mediation: 'conditional'` 字串呼叫 `navigator.credentials.get()` 方法，有條件地觸發螢幕鎖定 UI。

- 如要啟用條件式 UI，請在 `view/index.html` 檔案中的相關註解後，將現有的使用者名稱輸入欄位替換為下列 HTML：

view/index.html

```
<!-- TODO: Add passkeys to the browser autofill: Enable conditional UI. -->
<mdui-text-field id="username" label="Username" name="username" autocomplete="username
webauthn" autofocus></mdui-text-field>
```

偵測功能、叫用 WebAuthn，以及啟用條件式 UI

1. 在 `view/index.html` 檔案中，於相關註解後方，將現有的 `import` 陳述式替換成下列程式碼：

view/index.html

```
// TODO: Add passkeys to the browser autofill: Detect features, invoke WebAuthn, and enable a
conditional UI.
import {
  $,
  _fetch,
  loading,
  authenticate
} from "/client.js";
```

這段程式碼會匯入您先前實作的 `authenticate()` 函式。

2. 確認 `window.PublicKeyCredential` 物件可用，且 `PublicKeyCredential.isConditionalMediationAvailable()` 方法會傳回 `true` 值，然後呼叫 `authenticate()` 函式：

view/index.html

```
// TODO: Add passkeys to the browser autofill: Detect features, invoke WebAuthn, and enable a
conditional UI.
if (window.PublicKeyCredential &&
    PublicKeyCredential.getClientCapabilities) {
  try {

    // Is conditional UI available in this browser?
    const capabilities = await PublicKeyCredential.getClientCapabilities();
    if (capabilities.conditionalGet) {

      // If conditional UI is available, invoke the authenticate() function.
      const user = await authenticate();
      if (user) {

        // Proceed only when authentication succeeds.
        $("#username").value = user.username;
        loading.start();
        location.href = "/home";
      } else {
        throw new Error("User not found.");
      }
    }
  } catch (e) {
    loading.stop();

    // A NotAllowedError indicates that the user canceled the operation.
    if (e.name !== "NotAllowedError") {
      console.error(e);
      alert(e.message);
    }
  }
}
```

查看本節的解決方案程式碼

[view/index.html](#)

```
<!-- TODO: Add passkeys to the browser autofill: Enable conditional UI. -->
<mdui-text-field id="username" label="Username" name="username" autocomplete="username
webauthn" autofocus></mdui-text-field>
```

[view/index.html](#)

```
// TODO: Add passkeys to the browser autofill: Detect features, invoke WebAuthn, and enable a conditional UI.
import {
  $,
  _fetch,
  loading,
  authenticate
} from '/client.js';
```

view/index.html

```
// TODO: Add passkeys to the browser autofill: Detect features, invoke WebAuthn, and enable a conditional UI.

// Is WebAuthn available on this browser?
if (window.PublicKeyCredential &&
    PublicKeyCredential.getClientCapabilities) {
  try {
    // Is conditional UI available in this browser?
    const capabilities = await PublicKeyCredential.getClientCapabilities();
    if (capabilities.conditionalGet) {
      // If conditional UI is available, invoke the authenticate() function.
      const user = await authenticate();
      if (user) {
        // Proceed only when authentication succeeds.
        $('#username').value = user.username;
        loading.start();
        location.href = '/home';
      } else {
        throw new Error('User not found.');
```

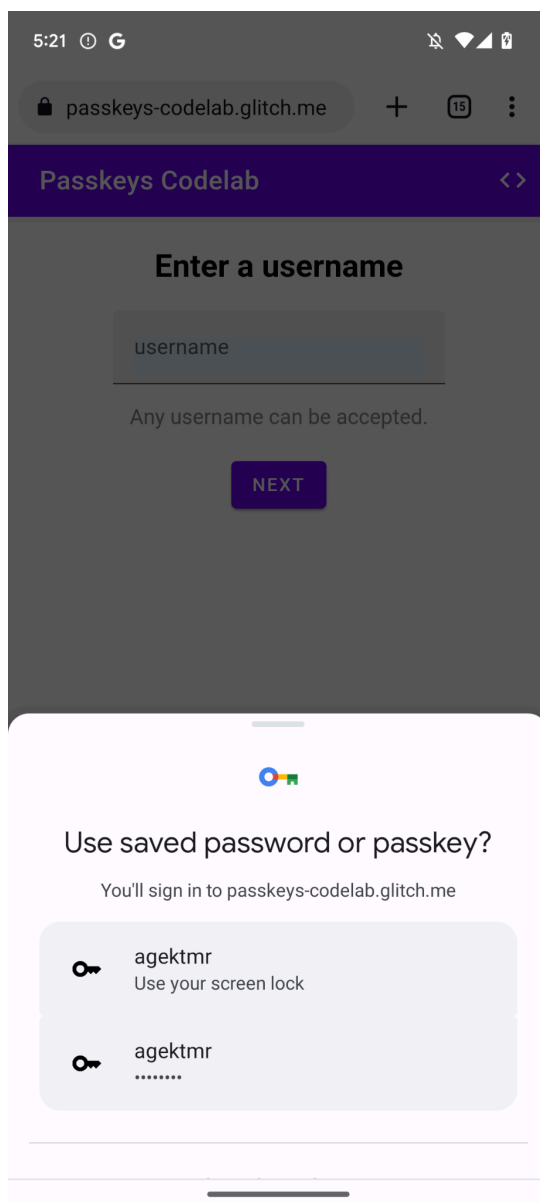
立即體驗

您已在網站上實作密碼金鑰的建立、註冊、顯示和驗證功能。

如要試用這項功能，請按照以下步驟操作：

1. 前往「預覽」分頁。
2. 視需要登出帳戶。

3. 按一下使用者名稱文字方塊。對話方塊隨即出現。
4. 選取要登入的帳戶。
5. 使用裝置的螢幕鎖定功能驗證身分。系統會將你重新導向至 `/home` 頁面並登入。



步驟 7 · 約 1 分鐘

7. 恭喜！

你已完成本程式碼研究室！如有任何疑問，請前往 [FIDO-DEV 郵寄清單](#) 提問，或在 [StackOverflow](#) 提問並加上 `passkey` 標記。

瞭解詳情

- [建立密碼金鑰，享有不需要密碼的登入體驗](#)
 - [透過表單自動填入功能使用密碼金鑰登入](#)
 - [使用密碼金鑰登入，不必輸入密碼](#)
-